

An Adaptive Tree Backup Algorithm for Nonlinear Function Approximation

Brett Daley

Northeastern University
Boston, Massachusetts 02115
daley.br@northeastern.edu

Abstract

Adaptive Tree Backup (ATB) algorithms have been recently proposed as a means of intelligently balancing bias and variance in temporal-difference learning by dynamically adjusting weighted backups on the basis of past data (Daley and Chan 2022); however, the extent to which these algorithms are useful in a function-approximation setting is presently unknown. In this report, we introduce *Action-Density* ATB, a deep reinforcement learning method that relies on an auxiliary maximum-likelihood model to implement adaptive backups in high-dimensional MDPs. We combine Action-Density ATB with a Deep Q-Network (DQN) and test its performance in a variety of stochastic Atari games. The proposed method can sometimes outperform the DQN baseline, and is the first demonstration of an ATB method learning successfully from high-dimensional sensory inputs.

1 Introduction

Unifying temporal-difference (TD), dynamic programming (DP), and Monte Carlo (MC) methods has helped illuminate trade-offs in reinforcement learning and has led to the development of better algorithms. Much prior work has focused on the *length* of backups as a unifying axis between TD and MC methods, in which these two classes can be understood as opposite extremes of the TD(λ) algorithm (Sutton 1988; Sutton and Barto 1998). In this view, the decay parameter $\lambda \in [0, 1]$ serves not only as a conceptual link between these distinct classes, but also as a mechanism for managing the bias-variance trade-off (Kearns and Singh 2000), where intermediate values of λ often deliver the best empirical performance (Sutton and Barto 1998).

Recent research has started to explore an orthogonal unification between TD and DP methods by varying the *width* of backups. This concept was formally introduced by Sutton and Barto (2018) in the $Q(\sigma)$ algorithm, which interpolates linearly between Expected Sarsa ($\sigma = 0$) (Sutton and Barto 1998) and Sarsa ($\sigma = 1$) (Rummery and Niranjan 1994). Daley and Chan (2022) recently extended this idea to time-varying weighted backups that evolve during training, producing a class of temporal-difference methods called Adaptive Tree Backup (ATB). The authors hypothesize that effective ATB methods should adjust the backup width based

on the number of visitations to each state-action pair, with more emphasis placed on frequently sampled pairs. They propose two methods called Count-Based ATB and Policy-Based ATB. These methods have the advantage of eliminating any user-specified hyperparameters, since they adapt automatically to the data experienced by the agent.

Although ATB methods are favorable to $Q(\sigma)$ in tabular environments, they appear to lack an easy extension to high-dimensional environments where state-action pairs are rarely revisited. In these instances, tracking visitation counts becomes ineffective. The principal goal of this project is therefore to investigate the possibility of an ATB method for use with large-scale, nonlinear function approximation (e.g. convolutional neural networks). Our main contribution is an algorithm that we call *Action-Density* ATB. Unlike other pseudocount-based methods designed for function approximation, which focus on state-density estimation, we propose a simpler strategy in which an auxiliary network is trained to predict the density of greedy actions with respect to a Deep Q-Network (DQN) (Mnih et al. 2015). We show that this strategy is mathematically related to optimal control and maximum-likelihood estimation. We test density-based ATB in a number of stochastic Atari games, where it can learn successful control policies and occasionally outperform a standard DQN baseline. Our experiments confirm that ATB methods can be effective in high-dimensional MDPs, and are not limited to explicit count-based methods.

2 Preliminaries

Reinforcement learning studies an agent interacting with its environment in order to maximize its expected cumulative reward (Sutton and Barto 2018). As is common in the literature, we model the agent-environment interface as a Markov Decision Process (MDP) determined by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$. Here, $\mathcal{S}$ is a finite set of environment states, $\mathcal{A}$ is a finite set of actions, $P(s'|s, a)$ is the transition probability function, and $R(s, a)$ is a scalar reward function. At each timestep t , the agent selects an action $a_t \in \mathcal{A}$ given the environment’s current state $s_t \in \mathcal{S}$, for which it receives the reward $R(s_t, a_t)$. The environment’s state subsequently changes to a new state s_{t+1} with probability $P(s_{t+1}|s_t, a_t)$.

The agent’s goal is to derive a policy $\pi(a|s)$ that maximizes its expected discounted return $\mathbb{E}_\pi[\sum_{k=0}^{\infty} \gamma^k r_k \mid s, a]$, where $\gamma \in [0, 1]$ is a discount factor that determines the

agent’s preference for future rewards. The solution to this problem is generally difficult, because even when an action in a given state appears to be initially rewarding, it may have long-term consequences on the environment state that ultimately renders it suboptimal.

Many algorithms for reinforcement learning generate estimates for the expected returns, in order to iteratively improve a policy until optimality. Value-based reinforcement learning agents iteratively refine an estimate $Q: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ of the expected discounted return $Q^\pi(s, a)$ earned by the policy π . Fundamental to the theory of these agents is the Bellman functional equation (Bellman 1966):

$$Q(s, a) = R(s, a) + \gamma \mathbb{E}_{s'} \left[\sum_{a'} \pi(a'|s') Q(s', a') \mid s, a \right]. \quad (1)$$

The Bellman equation has a unique solution, denoted by $Q^\pi(s, a)$, which coincides with the expected discounted return under the policy π . Estimating this quantity is useful because a greedy policy—defined as $\pi_g(a|s) = 1$ if and only if $a = \operatorname{argmax}_{a'} Q(s, a')$ and $\pi_g(a|s) = 0$ otherwise—is known to be a better policy than π by the policy improvement theorem (Sutton and Barto 1998).

Related to this is the Bellman optimality functional equation:

$$Q(s, a) = R(s, a) + \gamma \mathbb{E}_{s'} \left[\max_{a'} Q(s', a') \mid s, a \right]. \quad (2)$$

This differs from the Bellman equation by only the replacement of the expectation over the action probabilities with a maximization over the actions. In fact, the Bellman optimality equation can be viewed as a specific case of the Bellman equation in which the policy π is greedy with respect to Q . In this case, the equation admits a fixed-point solution known as Q^* , which represents the expected returns under an optimal policy π^* . As such, this equation is of great interest in control theory, and serves as a foundation for important methods in optimal behavior such as Q-Learning (Watkins 1989).

Although the Bellman equations can be solved exactly by dynamic programming techniques like value iteration (Bellman 1966), these techniques require explicit knowledge of the transition model P , which is often assumed to be unavailable to the agent. Alternatively, temporal-difference methods (Sutton 1988) utilize iterative stochastic approximation to generate the solution to the Bellman equations from sampled experience. Let $\alpha_t > 0$ be a time-dependent learning rate. This technique gives rise to

$$Q(s_t, a_t) \leftarrow (1 - \alpha_t) Q(s_t, a_t) + \alpha_t \left(r_t + \gamma \sum_{a'} \pi(a'|s_{t+1}) Q(s_{t+1}, a') \right), \quad (3)$$

which is known as either Expected Sarsa (Sutton and Barto 1998) or Tree Backup (Precup, Sutton, and Singh 2000) (these are the same for the 1-step update presented here).

When the explicit policy $\pi(a|s)$ is unknown, a sampled action $a_{t+1} \sim \pi(\cdot|s)$ can be used to remove the dependency on the policy probabilities:

$$Q(s_t, a_t) \leftarrow (1 - \alpha_t) Q(s_t, a_t) + \alpha_t (r_t + \gamma Q(s_{t+1}, a_{t+1})). \quad (4)$$

This method is known as Sarsa (Rummery and Niranjan 1994). Notice that Equations (3) and (4) are equivalent in expectation; in both cases, these algorithms are expected to converge to Q^π when the learning rate α_t is annealed towards zero at an appropriate rate. We may ask, then, what other types of weighted backups are convergent, and do they lead to faster convergence in practice? We explore these questions in the next section.

3 Adaptive Tree Backup Algorithms

Adaptive Tree Backup (ATB) algorithms (Daley and Chan 2022) are a form of temporal-difference learning (Sutton 1988), which gradually generates the on-policy value function for an MDP via bootstrapped sample interactions with the environment. The name ATB comes from the fact that these algorithms generalize Tree Backup (Precup, Sutton, and Singh 2000) to arbitrary coefficients $c(s, a)$ that replace $\pi(a|s)$ in Equation (3). The Bellman-like equation for ATB therefore becomes

$$Q(s, a) = R(s, a) + \gamma \mathbb{E}_{s'} \left[\sum_{a'} c(s', a') Q(s', a') \mid s, a \right], \quad (5)$$

subject to $\sum_{a' \in \mathcal{A}} c(s', a') = 1.$

where each $c(s', a')$ is a nonnegative (possibly random) coefficient.

An exact MDP model would be needed to perform dynamic-programming updates using Equation (5). By introducing a learning rate, we can make an incremental method suitable for stochastic approximation in MDPs. For a given transition (s_t, a_t, r_t, s_{t+1}) , temporal-difference ATB algorithms can be written in the generic form

$$Q(s_t, a_t) = (1 - \alpha_t) Q(s_t, a_t) + \alpha_t \left(r_t + \gamma \sum_{a'} c_t(s_{t+1}, a') Q(s_{t+1}, a') \right) \quad (6)$$

Note that the backup associated with update (5) admits the correct fixed point Q^π only if $\mathbb{E}[c_t(s, a)] = \pi(a|s)$. Even so, if we ensure that $\mathbb{E}[c_t(s, a)] \rightarrow \pi(a|s)$ as $t \rightarrow \infty$, then the ATB method will eventually converge to Q^π .

Daley and Chan (2022) observed that ATB learns well initially with sparse updates (where $c(s, a) = 0$ for most (s, a)) in order to circumvent bias in the initialization of Q ; however, as Q becomes more accurate with respect to the expected returns, the coefficients should revert back to Tree Backup (3) to ensure proper convergence and reduce variance. If their hypothesis is correct, then methods that

Algorithm 1: Tabular Adaptive Tree Backup (ATB) Methods

```
Input policy  $\pi$  and sequence of stepsizes  $\alpha_t \in [0, 1]$ 
Initialize  $Q(s, a)$  arbitrarily and  $n(s, a) = 0, \forall (s, a) \in \mathcal{S} \times \mathcal{A}$ 
Initialize environment state  $s_0$ 
for  $t = 0, 1, 2, \dots$  do
  Take action  $a_t \sim \pi(\cdot|s_t)$ , observe reward  $r_t$  and next state  $s_{t+1}$ 
   $n(s_t, a_t) \leftarrow n(s_t, a_t) + 1$ 

  For each  $a' \in \mathcal{A}$ , let  $c(a') = \begin{cases} n(s_{t+1}, a') & \text{(Count-Based)} \\ u(n(s_{t+1}, a'))\pi(a'|s_{t+1}) & \text{(Policy-Based)} \end{cases}$ 

  Let  $K = \sum_{a' \in \mathcal{A}} c(a')$ 
  For each  $a' \in \mathcal{A}$ , normalize  $c(a') \leftarrow \frac{1}{K} c(a')$ , where  $\frac{0}{0} = 0$ 
   $Q(s_t, a_t) \leftarrow (1 - \alpha_t)Q(s_t, a_t) + \alpha_t (r_t + \gamma \sum_{a' \in \mathcal{A}} c(a')Q(s_{t+1}, a'))$ 
end for
```

dynamically increase the effective backup width based on state-action pair visitations should generally perform better than fixed algorithms such as Tree Backup. We discuss their two proposed variants that do this, and then introduce a third (novel) variant that is compatible with function approximation.

3.1 Count-Based ATB

To match the intuition that an effective ATB strategy should ignore unvisited state-action pairs, we can weight state-action pairs according to their relative frequency of appearance. This can be accomplished by tracking a count $n(s, a)$ for each state-action pair (s, a) and then normalizing it as a proportion. This variant, which is called Count-Based ATB (Daley and Chan 2022), defines the backup coefficients as

$$c_t(s, a) = \frac{n(s, a)}{\sum_{a'} n(s, a')}. \quad (7)$$

By the law of large numbers, $c_t(s, a) \rightarrow \pi(a|s)$ after infinitely many visitations to state s , and the method should converge to Q^π in the limit. Beyond convergence, there are several reasons why this particular ATB formulation is appealing. First, the count-based backup amounts to a maximum-likelihood 1-step estimate of $Q(s, a)$, and in this sense is the “most reasonable” backup to perform based on previously observed experiences. Furthermore, frequency counts implicitly emphasize value estimates that are more reliable, while excluding those that have not changed since initialization. This means that Count-Based ATB automatically transitions from Sarsa to Expected Sarsa, with a different effective rate for each state, and which does not require a user-specified hyperparameter schedule.

Unfortunately, these benefits are not without some drawbacks. Although the algorithm theoretically converges to Q^π in the limit, it is extremely unlikely that $c(s, a) = \pi(a|s)$ after any finite amount of training. This means that Count-Based ATB may tend towards a fixed point other than Q^π in practice, which is undesirable even if it does represent a maximum-likelihood estimate according to past experience. Another problem—which is much more significant—occurs when the agent switches its policy from π to a different policy π' during training. At this point, all of the previously

collected counts $n(s, a)$ will no longer reflect the correct on-policy distribution. The agent would either need to reset the counts to zero (thus discarding useful information), or wait many timesteps for the counts to accurately reflect the new policy π' . Both of these options are inefficient, motivating us to search for a better alternative.

3.2 Policy-Based ATB

When the agent’s policy is known explicitly, we can resolve the issues of Count-Based ATB by directly utilizing knowledge of the action probabilities $\pi(a|s)$. This is commonly done in temporal-difference learning (e.g. Expected Sarsa and Tree Backup). If some state-action pair (s, a) is visited for the first time, then we know that the eventual value of $c_t(s, a)$ will be $\pi(a|s)$ according to Count-Based ATB. We can greatly accelerate convergence by immediately assigning $c_t(s, a) \leftarrow \pi(a|s)$ after this first visitation. Let u denote the unit step function such that $u(n) = 1$ if $n > 0$ and $u(n) = 0$ otherwise. The backup coefficients for this new variant, which is called *Policy-Based* ATB (Daley and Chan 2022), become

$$c_t(s, a) = \frac{u(n(s, a))}{\sum_{a'} u(n(s, a'))} \pi(a|s). \quad (8)$$

This definition clearly solves the two problems of Count-Based ATB. Unvisited state-action pairs are still ignored, as desired, but after each pair has been sampled at least once, the algorithm becomes equivalent to Expected Sarsa and starts converging to Q^π . This effectively bypasses the infinite-visitation requirement of Count-Based ATB. We can also see that the weighted backup instantly reflects changes to the policy, thanks to the explicit dependency on π in definition (8). For these reasons, Policy-Based ATB learns significantly faster than Count-Based ATB in a tabular setting; however, it may be harder to combine with function approximation in high-dimensional MDPs, where the need for generalization makes it difficult to determine whether a state should be considered visited or unvisited. Additionally, the reliance on an explicit policy model $\pi(a|s)$ may be restrictive in some settings.

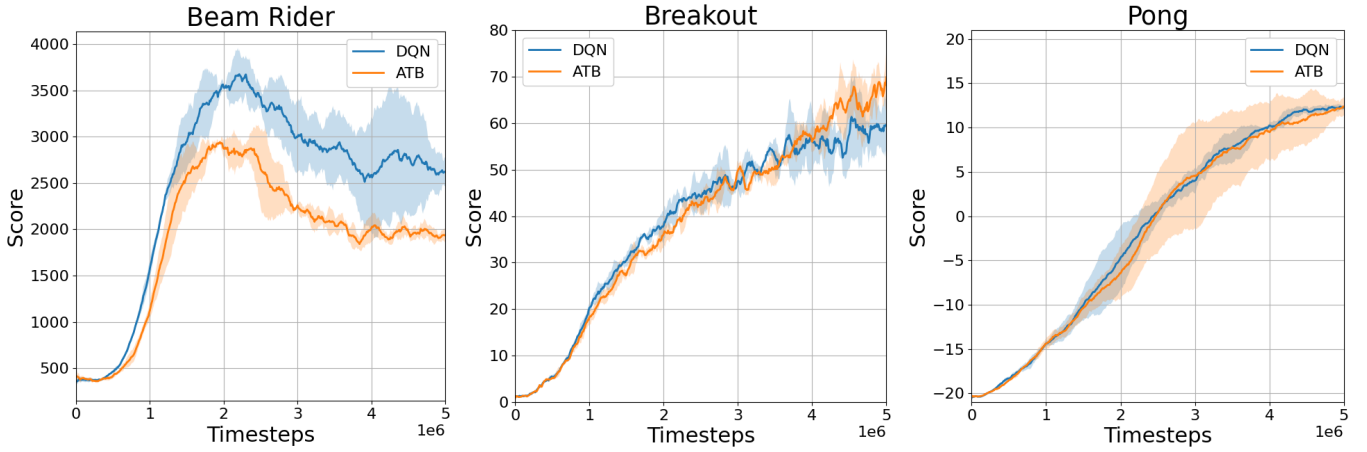


Figure 1: A comparison of Action-Density ATB and a baseline DQN implementation in 3 Atari 2600 games: Beam Rider, Breakout, and Pong. The x -axis is the number sample interactions with the environment, measured in timesteps. The y -axis is the 100-episode moving average of the undiscounted return (i.e. game score). Results are averaged over 3 trials for both algorithms; shaded areas represent one standard deviation.

3.3 Action-Density ATB

So far, the proposed ATB methods rely on state-action pair visitation counts, which restrict their applicability to high-dimensional MDPs. In the presence of rich sensory inputs such as images, it is astronomically unlikely that an agent ever sees the same input twice. Consequently, count-based methods become ineffective, since each experience will appear to be novel to the agent. Although there is a large body of work that has focused on extending count-based methods to the function-approximation setting, typically for the purposes of exploration (e.g. Bellemare et al. 2016), they are difficult to implement because they are not compatible with minibatched experience replay. Ideally, our method would minimize a loss function over minibatches sampled from a replay memory of past experiences, since this has become the dominant paradigm for deep reinforcement learning.

How else can we avoid the need for explicit counts or pseudocounts, such that learning in large-scale MDPs becomes feasible? We would still like a method that extends the intuition of Count-Based ATB by placing more emphasis on frequently-visited state-action pairs, but does so in a “fuzzy” way that does not depend on the exact number of visitations to a state-action pair. Towards this, we propose to estimate the *density* of greedy actions with respect to the value function Q . We call our method Action-Density ATB. We start from the Bellman optimality equation (2) and rewrite the maximization over state-action values as

$$\max_{a'} Q(s', a') = \sum_{a'} \mathbb{I}\{a' = \operatorname{argmax}_{a''} Q(s', a'')\} Q(s', a')$$

where $\mathbb{I}$ is the indicator function. Notice that, for a greedy policy, we trivially have

$$\begin{aligned} \mathbb{I}\{a' = \operatorname{argmax}_{a''} Q(s', a'')\} \\ = \Pr(a' = \operatorname{argmax}_{a''} Q(s', a'') | s'). \end{aligned}$$

This suggests, interestingly enough, that algorithms based on the Bellman optimality equation (e.g. Q-Learning) can

be expressed as an ATB method with

$$c(s, a) = \Pr(a = \operatorname{argmax}_{a''} Q(s, a'') | s). \quad (9)$$

In a tabular setting, there is no apparent benefit to this alternative formulation; however, now consider learning in a high-dimensional MDP where function approximation is necessary to generate the coefficients $c(s, a)$. In this case, $c(s, a)$ will most likely not be exactly 0 or 1, but instead a fractional value in between them, and the algorithm resembles ATB more closely. We also hypothesize that this will lead to lower-variance updates compared to Deep Q-Learning (Mnih et al. 2015) as a consequence. (Unfortunately, we do not have time to investigate this theoretically in this report.)

Crucially, because $c(s, a)$ represents a conditional probability distribution, we can train a model to predict the mapping

$$(s, a) \mapsto \Pr(a = \operatorname{argmax}_{a''} Q(s, a'') | s) \quad (10)$$

with maximum likelihood by minimizing a cross-entropy loss. This allows us to utilize minibatched experience replay, while also circumventing the need for generating semantically meaningful and accurate pseudocounts.

In practice, we must define the loss function in terms of a model architecture. We assume that the coefficients $c(s, a; \phi)$ are produced by a network, parameterized by a vector ϕ , whose output activation function is the softmax. Our loss $L(\phi)$ is defined as

$$L(\phi) = H(c(s, \cdot; \phi), \operatorname{onehot}(\operatorname{argmax}_a Q(s, a; \theta^-))), \quad (11)$$

where $H(p, q)$ is the cross entropy between categorical distributions p and q . Minimizing the cross entropy is equivalent to maximizing the likelihood for categorical classification (Goodfellow, Bengio, and Courville 2016).

The update for the value function, which we assume to be

parameterized by θ , then becomes

$$L(\theta) = \left(r_t + \gamma \sum_{a'} c(s', a'; \phi^-) Q(s_{t+1}, a'; \theta^-) - Q(s, a; \theta) \right)^2 \quad (12)$$

This is essentially the standard DQN loss, but the maximization is replaced by a sum over actions using the coefficients produced by the ϕ -network. The losses $L(\phi)$ and $L(\theta)$ are jointly minimized by a first order method optimization. In practice, we also introduce target networks parameterized by stale copies ϕ^- and θ^- , which are copied periodically from ϕ and θ , respectively, to help promote the stability of the algorithm (Mnih et al. 2015).

4 Experiments

We test the performance of Action-Density ATB in three Atari 2600 games from the Arcade Learning Environment (Brockman et al. 2016). We picked Beam Rider, Breakout, and Pong; these were chosen because they are games that are known to be easily learned by DQN, yet also have reasonable variation in terms of high-level objective and reward sparsity. (More games would be tested ideally, but we unfortunately did not have time.) Importantly, we applied random action repeat (Machado et al. 2018) with $\xi = 0.25$ to ensure the robustness in stochastic environments by preventing the agents from simply memorizing high-reward trajectory. Our baseline is a standard implementation of DQN, which serves as a suitable comparison to Action-Density ATB because they are both mathematically based upon Q-Learning. Therefore, by changing only the loss definition of DQN, we are able to effectively isolate the impact of adaptive coefficients in the learning process.

For the value function $Q(s, a; \theta)$, we adopted the neural architecture from (Mnih et al. 2015): i.e. three convolutional layers followed by one ReLU hidden layer of 512 units, and a final linear mapping of size $|\mathcal{A}|$. For the density model $c(s, a; \phi)$ used by ATB, we duplicated this architecture and added a softmax activation to the final linear mapping. Images were resized to 84×84 pixels and converted to grayscale, and the previous four images were fed as the input (i.e. the MDP “state” s) to the neural networks. We clipped rewards, as did (Mnih et al. 2015); however, we did not treat a lost life as episode termination, per the recommendation in (Machado et al. 2018). The networks were trained by Adam (Kingma and Ba 2014) with a learning rate of 5×10^{-5} on minibatches of size 32 sampled uniformly from a replay memory with capacity 1000000 transitions.

Learning curves for the ATB and DQN agents are plotted in Figure 1 over the course of training for 5 million timesteps. The performance metric is the game score achieved by the agents during training, smoothed by a 100-episode average, plotted against the training time. We conducted 3 trials for both agents, averaged the results, and shaded the standard deviation. ATB shows some slight improvement in Breakout towards the end of training compared to DQN, but performed almost the same in Pong, and obtained a much lower score in Beam Rider (though both agents appeared to lose performance at the end). It is possi-

ble that the improvement in Breakout would be greater with additional training, but we cannot conclusively say this from the data presented here. In the end, ATB does not appear to be a clear improvement over DQN—at least based on these results—but does show potential for learning control policies in large-scale environments. We discuss reasons this may be the case in the next section, as well as possibilities for achieving better performance in future work.

5 Discussion

Although Action-Density ATB does not unequivocally improve the performance of DQN in these preliminary experiments, we have successfully verified that Adaptive Tree Backup methods can be combined with deep neural networks and minibatched experience replay to learn effective control policies. Even in the case where ATB does not outperform DQN (Beam Rider), the results still demonstrate that the agent is capable of learning a strategy to obtain a high score on the game. As such, we have confirmed one of the hypotheses of this paper, which is that greedy-action density estimation can be used to avoid explicitly tracking pseudocounts. We therefore anticipate that future work would be able to enhance our proposed algorithm and obtain more-competitive results.

We believe that the worse performance in Beam Rider, and the similar but higher-variance performance in Pong, is due to the additional complexity of training the second network $c(s, a; \phi)$. We suspect that inaccuracies in the network estimation are compounded by the mutual dependency between the value function Q . Specifically, any small errors in the minimization of Equation (11) (perhaps due to incomplete training) would impact the weighted backup used in the construction of Equation (12). In turn, this would produce a poor value function, which on the next epoch would set up an incorrect maximum-likelihood estimation, and so on. To some extent, this phenomenon should be mitigated—at least partially—by the target networks we employed, but it is still possible that the parameters ϕ^- could fail to represent a good maximum-likelihood estimate with respect to θ^- . We therefore expect that training the density network more relative to the value network could improve performance. Currently, a minibatch of training is conducted for both networks every 4 timesteps, but this could be separated into two distinct hyperparameters: e.g. by training the density network every 2 timesteps. It would be interesting to see how these values affect learning, although we did not have a chance to experiment with this.

We note that ATB uses four networks instead of two (compared to DQN) because of its additional density network and target network. In light of the potential issues discussed in the previous paragraph, we speculate that the target network ϕ^- for density estimation could be removed entirely. By eliminating the stale parameters, it could be possible to improve the “synchronization” between the maximum-likelihood density estimation and the current value function. This could potentially be because the target network θ^- for the value function would serve to stabilize both of the losses indirectly. Again, we did not have time to experiment with this possibility, but we believe that it should be explored in

future work—both in the interest of improved empirical performance and decreased computational requirements.

One final explanation for the inconclusive results—although one that we believe to be false—may be that Action-Density ATB is too similar to Q-Learning to produce a meaningful improvement. Equations (11) and (12) are equal in expectation, and it would not be impossible that they produce the same results on average because of this. However, a counterpoint to this is that we do see that the results are statistically significant in Figure 1, showing that there is substantial variation in the behavior of the methods that is likely not explained by random chance. In this view, we can reasonably conclude that ATB does perform differently from DQN, even if it is unclear whether the effect is always beneficial.

Ultimately, we were able to show that ATB backups can be effective with deep neural networks, which was the principal goal of this project. It may turn out that tweaks are needed to the hyperparameters, or a different formulation would be more appropriate. Many interesting avenues for future work remain open.

6 Conclusion

ATB methods offer many new strategies to manage bias and variance in temporal-difference learning, while also unifying existing methods in a common mathematical form. Previously, ATB methods were limited to count-based implementations, which precluded their straightforward extension to learning from high-dimensional sensory inputs. Our newly proposed Action-Density ATB is the first ATB method to be easily compatible with nonlinear function approximation. It achieves this by recasting Q-Learning as a maximum-likelihood estimation problem, offering an interesting semantic interpretation and compelling rationale for ATB in optimal control problems while also permitting an easy implementation with minibatched experience replay. Although our preliminary experiments here indicate that further work is needed to make the algorithm consistently outperform DQN, they are promising in the sense that they show ATB strategies can be scalable to high-dimensional problems of great interest in the research community.

References

Bellemare, M.; Srinivasan, S.; Ostrovski, G.; Schaul, T.; Saxton, D.; and Munos, R. 2016. Unifying count-based exploration and intrinsic motivation. *Advances in neural information processing systems*, 29.

Bellman, R. 1966. Dynamic Programming. *Science*, 153(3731): 34–37.

Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. Openai gym. *arXiv preprint arXiv:1606.01540*.

Daley, B.; and Chan, I. 2022. Adaptive Tree Backup Algorithms for Temporal-Difference Reinforcement Learning. In *Reinforcement Learning and Decision Making*, volume 5.

Goodfellow, I.; Bengio, Y.; and Courville, A. 2016. *Deep learning*. MIT press.

Kearns, M. J.; and Singh, S. P. 2000. Bias-Variance Error Bounds for Temporal Difference Updates. In *COLT*, 142–147.

Kingma, D. P.; and Ba, J. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Machado, M. C.; Bellemare, M. G.; Talvitie, E.; Veness, J.; Hausknecht, M.; and Bowling, M. 2018. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61: 523–562.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540): 529–533.

Precup, D.; Sutton, R. S.; and Singh, S. 2000. Eligibility Traces for Off-Policy Policy Evaluation. In *Proceedings of the Seventeenth International Conference on Machine Learning*, 759–766.

Rummery, G. A.; and Niranjan, M. 1994. On-Line Q-Learning Using Connectionist Systems. Technical Report TR 166, Cambridge University Engineering Department, Cambridge, England.

Sutton, R. S. 1988. Learning to Predict by the Methods of Temporal Differences. *Machine Learning*, 3(1): 9–44.

Sutton, R. S.; and Barto, A. G. 1998. *Reinforcement Learning: An Introduction*. MIT Press, 1st edition.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.

Watkins, C. J. C. H. 1989. *Learning from Delayed Rewards*. Ph.D. thesis, King’s College, Cambridge.